Problema 2 (2.5 p)

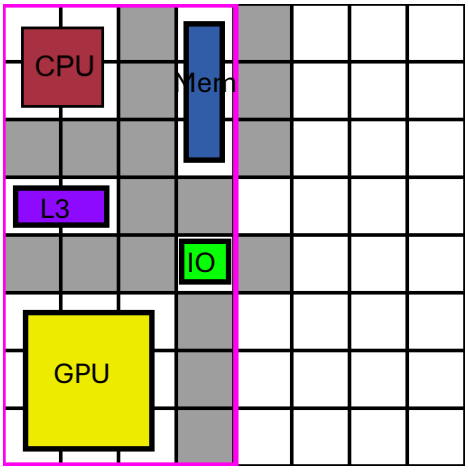
Queremos configurar un procesador de tipo System On A Chip (SOC) de forma modular para un dispositivo de alto rendimiento multimedia y juegos. Estos procesadores incluyen la mayoría de los componentes que tiene un computador moderno. Disponemos de un área máxima disponible de 8 x 8 casillas (64 casillas) para colocar estos componentes que son los siguientes:

- CPU: Ocupa 2x2 = 4 casillas
- Interfaz de memoria: Ocupa 3 x 1 = 3 casillas
- Controladora de IO: Ocupa 1 x 1 = 1 casilla
- GPU: Ocupa 3x3 = 9 casillas
- Memoria Cache L3 compartida entre CPU y GPU ocupa 1x2 = 2 casillas

Pero hay que cumplir ciertas restricciones:

- La memoria cache L3 es deseable que esté lo más cerca posible de CPU y GPU.
- Tiene que haber una casilla de separación entre cada componente para poder colocar el bus de conexión entre ellos y para que el calor que genere cada componente no se propague al resto. Las diagonales no cuentan, solo en horizontal y vertical.

El objetivo es minimizar el área ocupada efectiva para poder construir procesadores lo más pequeños posibles y así poder obtener más chips por oblea. El área efectiva **es el área rectangular mínima** que engloba a todos los componentes del SOC. En el ejemplo está representado como un cuadrado rosa. **Los componentes pueden colocarse en horizontal o en vertical pero no en diagonal.**



Nota: La parte gris en el dibujo muestra las zonas donde no puede colocarse otro chip.

Se pide:

1. Elegir uno de los métodos vistos en clase para resolver el problema y explica razonadamente por qué lo has elegido (1p)
2. Modelar el problema usando el método o algoritmo seleccionado. Recordad que, en función del tipo de algoritmo, se necesita una modelización diferente. (1.5P)

Se pide:

3. Elegir uno de los métodos vistos en clase para resolver el problema y explica razonadamente por qué lo has elegido (1p)

El formalismo que mejor encaja sería algoritmos genéticos porque es un problema de optimización, pero también podría plantearse como un algoritmo de búsqueda. Si lo planteamos como algoritmo de búsqueda sería:

- Estado inicial: la matriz vacía
- Estado final: la matriz con todos los elementos
- Operadores: colocar CPU, GPU, L3, etc. cada uno tendría un coste que sería su área más el espacio alrededor suyo sin contar las diagonales.

4. Modelar el problema usando el método o algoritmo seleccionado. Recordad que, en función del tipo de algoritmo, se necesita una modelización diferente. (1.5P)

Modelado usando genéticos:

La representación de los individuos sería la posición (X, Y) de cada una de las piezas, además de la rotación en el caso de la de L3 y Mem (0 en horizontal y 1 en vertical). Habría que fijar el punto donde se establece la posición en cada una de ellas. Por ejemplo, en la esquina superior izquierda.

CPU{X,Y	GPU{X,Y	IO{X,Y}	MEM{X,Y,O}	L3{X,Y,O}

El ejemplo del enunciado sería:

[0,0] [0,5] [3,4][3,0,1] [0,3,0]

O se hace una codificación en binario o si se mantiene una codificación decimal, entonces hay que tener en cuenta que la mutación de los genes que representan la dirección solo puede tomar los valores 0-1. En los cruces no hay que descolocar los alelos para evitar problemas ya que no todos significan lo mismo.

¿Cuál es la función de fitness? pues habría que comprobar el área que engloba cada pieza, por ejemplo, marcándola en la matriz y después quedándonos con los valores más altos y bajos de x e y, y calcular su resta para saber el tamaño del rectángulo en horizontal y vertical. Después aplicando el área de un rectángulo podremos saber el área ocupada por los componentes de la gpu. Para premiar que la cache L3 se sitúe a 1 casilla de distancia de la GPU y CPU (que es lo mínimo permitido) aumentaremos el fitness de aquellos que tengan la Cache L3 a más distancia de 1 de forma proporcional a la distancia a ambos componentes. Otra opción es usar este criterio como desempate. Priorizando el área mínima primero.

¿Qué pasa si generamos ejemplos inválidos?

Debemos gestionar que sucede si colocamos en la mutación o en el cruce a dos elementos que no pueden ser colocados. Hay diferentes alternativas, por un lado

- Podemos penalizar con un fitness muy bajo a los individuos inválidos para que así no sean seleccionados por el algoritmo
- Podemos intentar que en la mutación no se generen valores inválidos si llevamos la cuenta de en qué casillas se permite la colocación de los chips y en el caso de elegir un valor inadecuado (no quepa el chip en esa área o caiga en una casilla contigua) repetir la elección hasta que quepa en alguna ubicación.

Otra representación posible seria numerar las casillas de la 1 a la 64 e indicar la casilla en la que va la pieza correspondiente.

Representaciones como usar la matriz de 64 casillas para representar un individuo o usar todas las casillas que ocupan los chips (4 en la cpu por ejemplo) para representar el individuo, tiene como problema un mayor número de individuos inválidos. Podríamos generar en el último caso, individuos que tengan una forma no adecuada al componente o en el caso de usar como individuo la matriz de 64 casillas, poner 2 cpus. Con la codificación elegida esto no sucede. Aun así, no parece que exista una codificación más o menos obvia que no de ningún tipo de error.